

7

模板与泛型编程

Templates and Generic Programming

C++ templates 的最初发展动机很直接：让我们得以建立“类型安全”（type-safe）的容器如 vector, list 和 map。然而当愈多人用上 templates，他们发现 templates 有能力完成愈多可能的变化。容器当然很好，但泛型编程（generic programming）——写出的代码和其所处理的对象类型彼此独立——更好。STL 算法如 for_each, find 和 merge 就是这一类编程的成果。最终人们发现，C++ template 机制自身是一部完整的图灵机（Turing-complete）：它可以被用来计算任何可计算的值。于是导出了模板元编程（template metaprogramming），创造出“在 C++ 编译器内执行并于编译完成时停止执行”的程序。近来这些日子，容器反倒只成为 C++ template 馅饼上的一小部分。然而尽管 template 的应用如此宽广，有一组核心观念一直支撑着所有基于 template 的编程。那些观念便是本章焦点。

本章无法使你变成一个专家级的 template 程序员，但可以使你成为一个比较好的 template 程序员。本章也会给你必要信息，使你能够扩展你的 template 编程，到达你所渴望的境界。

条款 41：了解隐式接口和编译期多态

Understand implicit interfaces and compile-time polymorphism.

面向对象编程世界总是以显式接口（explicit interfaces）和运行期多态（runtime polymorphism）解决问题。举个例子，给定这样（无甚意义）的 class：

```
class Widget {  
public:  
    Widget();  
    virtual ~Widget();
```

```

    virtual std::size_t size( ) const;
    virtual void normalize();
    void swap(Widget& other);           //见条款 25
    ...
};

```

和这样（也是无甚意义）的函数：

```

void doProcessing(Widget& w)
{
    if (w.size() > 10 && w != someNastyWidget) {
        Widget temp(w);
        temp.normalize();
        temp.swap(w);
    }
}

```

我们可以这样说 doProcessing 内的 w：

- 由于 w 的类型被声明为 Widget，所以 w 必须支持 Widget 接口。我们可以在源码中找出这个接口（例如在 Widget 的 .h 文件中），看看它是什么样子，所以我称此为一个显式接口（*explicit interface*），也就是它在源码中明确可见。
- 由于 Widget 的某些成员函数是 virtual，w 对那些函数的调用将表现出运行期多态（*runtime polymorphism*），也就是说将于运行期根据 w 的动态类型（见条款 37）决定究竟调用哪一个函数。

Templates 及泛型编程的世界，与面向对象有根本上的不同。在此世界中显式接口和运行期多态仍然存在，但重要性降低。反倒是隐式接口（*implicit interfaces*）和编译期多态（*compile-time polymorphism*）移到前头了。若想知道那是什么，看看当我们将 doProcessing 从函数转变成函数模板（*function template*）时发生什么事：

```

template<typename T>
void doProcessing(T& w)
{
    if (w.size() > 10 && w != someNastyWidget) {
        T temp(w);
        temp.normalize();
        temp.swap(w);
    }
}

```

现在我们怎么说 doProcessing 内的 w 呢？

- w 必须支持哪一种接口，系由 **template** 中执行于 w 身上的操作来决定。本例看来 w 的类型 T 好像必须支持 size, normalize 和 swap 成员函数、copy 构造函数（用

以建立 `temp`)、不等比较 (*inequality comparison*, 用来比较 `someNasty-Widget`)。我们很快会看到这并非完全正确, 但对目前而言足够真实。重要的是, 这一组表达式 (对此 `template` 而言必须有效编译) 便是 `T` 必须支持的一组隐式接口 (*implicit interface*)。

- 凡涉及 `w` 的任何函数调用, 例如 `operator>` 和 `operator!=`, 有可能造成 `template` 具现化 (*instantiated*), 使这些调用得以成功。这样的具现行为发生在编译期。
“以不同的 `template` 参数具现化 `function templates`”会导致调用不同的函数, 这便是所谓的编译期多态 (*compile-time polymorphism*)。

纵使你从未用过 `templates`, 应该不陌生“运行期多态”和“编译期多态”之间的差异, 因为它类似于“哪一个重载函数该被调用” (发生在编译期) 和“哪一个 `virtual` 函数该被绑定” (发生在运行期) 之间的差异。显式接口和隐式接口的差异就比较新颖, 需要更多更贴近的说明和解释。

通常显式接口由函数的签名式 (也就是函数名称、参数类型、返回类型) 构成。

例如 `Widget` `class`:

```
class Widget {
public:
    Widget();
    virtual ~Widget();
    virtual std::size_t size() const;
    virtual void normalize();
    void swap(Widget& other);
};
```

其 `public` 接口由一个构造函数、一个析构函数、函数 `size`, `normalize`, `swap` 及其参数类型、返回类型、常量性 (*constnesses*) 构成。当然也包括编译器产生的 *copy* 构造函数和 *copy assignment* 操作符 (见条款 5)。另外也可以包括 `typedefs`, 以及如果你大胆违反条款 22 (令成员变量为 `private`) 而出现的 `public` 成员变量。

隐式接口就完全不同了。它并不基于函数签名式, 而是由有效表达式 (*valid expressions*) 组成。再次看看 `doProcessing` `template` 一开始的条件:

```
template<typename T>
void doProcessing( T& w)
{
    if (w.size() > 10 && w != someNastyWidget) {
        ...
    }
}
```

T (w 的类型) 的隐式接口看来好像有这些约束:

- 它必须提供一个名为 `size` 的成员函数, 该函数返回一个整数值。
- 它必须支持一个 `operator!=` 函数, 用来比较两个 T 对象。这里我们假设 `someNastyWidget` 的类型为 T。

真要感谢操作符重载 (`operator overloading`) 带来的可能性, 这两个约束都不需要满足。是的, T 必须支持 `size` 成员函数, 然而这个函数也可能从 `base class` 继承而得。这个成员函数不需返回一个整数值, 甚至不需返回一个数值类型。就此而言, 它甚至不需要返回一个定义有 `operator>` 的类型! 它唯一需要做的是返回一个类型为 `x` 的对象, 而 `x` 对象加上一个 `int(10)` 的类型必须能够调用一个 `operator>`。这个 `operator>` 不需要非得取得一个类型为 `x` 的参数不可, 因为它也可以取得类型 `y` 的参数, 只要存在一个隐式转换能够将类型 `x` 的对象转换为类型 `y` 的对象!

同样道理, T 并不需要支持 `operator!=`, 因为以下这样也是可以的: `operator!=` 接受一个类型为 `x` 的对象和一个类型为 `y` 的对象, T 可被转换为 `x` 而 `someNastyWidget` 的类型可被转换为 `y`, 这样就可以有效调用 `operator!=`。

(偷偷告诉你, 以上分析并未考虑这样的可能性: `operator&&` 被重载, 从一个连接词改变为或许完全不同的某种东西, 从而改变上述表达式的意义。)

当人们第一次以此种方式思考隐式接口, 大多数的他们会感到头疼。但真的不需要阿司匹林来镇痛。隐式接口仅仅是由一组有效表达式构成, 表达式自身可能看起来很复杂, 但它们要求的约束条件一般而言相当直接又明确。例如以下条件式:

```
if (w.size() > 10 && w != someNastyWidget) ...
```

关于函数 `size`, `operator>`, `operator&&` 或 `operator!=` 身上的约束条件, 我们很难就此说得太多, 但整体确认表达式约束条件却很容易。if 语句的条件式必须

是个布尔表达式, 所以无论涉及什么实际类型, 无论 `w.size() > 10 && w != someNastyWidget` 导致什么, 它都必须与 `bool` 兼容。这是 `template doProcessing` 加诸于其类型参数 (type parameter) `T` 的隐式接口的一部分。`doProcessing` 要求的其他隐式接口: `copy` 构造函数、`normalize` 和 `swap` 也都必须对 `T` 型对象有效。

加诸于 `template` 参数身上的隐式接口, 就像加诸于 `class` 对象身上的显式接口一样真实, 而且两者都在编译期完成检查。就像你无法以一种“与 `class` 提供之显式接口矛盾”的方式来使用对象 (代码将通不过编译), 你也无法在 `template` 中使用“不支持 `template` 所要求之隐式接口”的对象 (代码一样通不过编译)。

请记住

- `classes` 和 `templates` 都支持接口 (interfaces) 和多态 (polymorphism)。
- 对 `classes` 而言接口是显式的 (explicit), 以函数签名为中心。多态则是通过 `virtual` 函数发生于运行期。
- 对 `template` 参数而言, 接口是隐式的 (implicit), 莫基于有效表达式。多态则是通过 `template` 具现化和函数重载解析 (function overloading resolution) 发生于编译期。

条款 42: 了解 typename 的双重意义

Understand the two meanings of `typename`.

提一个问题: 以下 `template` 声明式中, `class` 和 `typename` 有什么不同?

```
template<class T> class Widget;           //使用 "class"  
template<typename T> class Widget;       //使用 "typename"
```

答案: 没有不同。当我们声明 `template` 类型参数, `class` 和 `typename` 的意义完全相同。某些程序员始终比较喜欢 `class`, 因为可以少打几个字。其他人 (包括我) 比较喜欢 `typename`, 因为它暗示参数并非一定得是个 `class` 类型。少数开发人员在接受任何类型时使用 `typename`, 而在只接受用户自定义类型时保留旧式的 `class`。然而从 C++ 的角度来看, 声明 `template` 参数时, 不论使用关键字 `class` 或 `typename`, 意义完全相同。

然而 C++ 并不总是把 `class` 和 `typename` 视为等价。有时候你一定得使用 `typename`。为了解其时机, 我们必须先谈谈你可以在 `template` 内指涉 (*refer to*) 的两种名称。